

ORM vs Native SQL (Query Builder / Raw SQL)

Feature	ORM (e.g., Sequelize, Prisma)	Native SQL / Query Builder (e.g., Knex, pg)
 Ease of Use	Very beginner-friendly. Write in JS/TS, not SQL.	Requires understanding SQL directly.
 Control	Abstracts away SQL—less control	Full control over queries and performance tuning
 Validation/Schema Sync	Built-in model validation, syncing	Manual schema setup, but more flexible
 Complex Queries	Can be difficult or inefficient	Easier to optimize complex joins, CTEs, raw queries
 Performance	Slightly slower due to abstraction	More performant with large data sets or custom queries
 Migrations	Built-in support (e.g., Sequelize CLI)	Knex/Manual or external tools
 Developer Experience	Excellent with auto-complete, models, TS support	Manual typing, but more control
 Ideal For	CRUD-heavy apps, MVPs, startups	Performance-critical apps, data-heavy platforms

What Do Big Companies Use?

Company	Approach	Notes
Netflix	Native SQL + Custom tools	High performance & custom infra
Airbnb	Sequelize (formerly), then moved to raw SQL in some parts	Switched for better control
Instagram (Meta)	Raw SQL with migrations	Data-heavy, highly optimized
Shopify	ORM (ActiveRecord for Rails)	Easy maintainability
GitHub	ORM (ActiveRecord)	Legacy monolith

Company	Approach	Notes
Uber	Mostly raw SQL	Custom infrastructure
Reddit	Native + some ORM	Depends on service

✅ When to Use ORM?

- You're building an MVP or internal tool
- You want to move fast with clear models and DB syncing
- You prefer DX and don't want to write raw SQL

✅ When to Use Native SQL?

- You need **maximum performance**
 - You work with **complex joins, transactions**
 - You want **full DB control**
 - You're scaling to millions of records
-

🧠 Personal Suggestion for You